

A BEGINNER'S GUIDE TO

Building a Bluetooth Low Energy Peripheral on Linux

Understanding BlueZ, D-Bus, GATT, and a real C project
from the very first line to the last

What you will learn

- What Bluetooth Low Energy actually is, in plain language
- How Linux talks to a Bluetooth chip through BlueZ and D-Bus
- How this project's ten source files fit together
- How a phone app like nRF Connect "sees" and talks to this code

No prior Bluetooth or D-Bus knowledge required.

Table of Contents

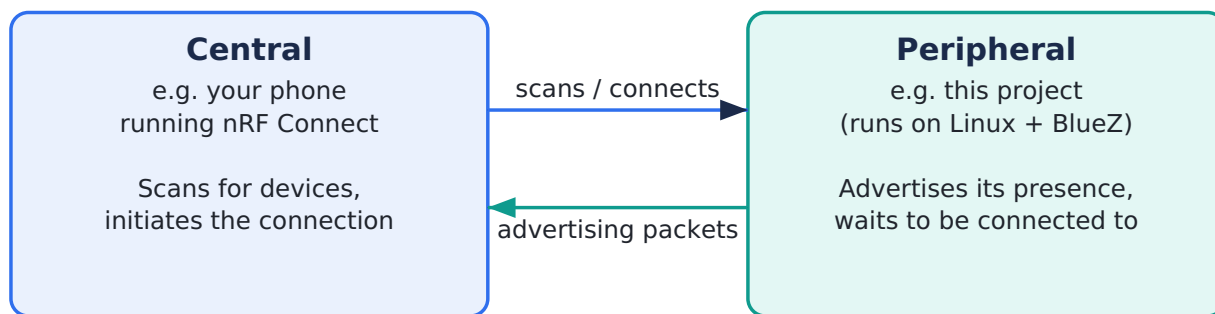
Chapter 1	Introduction: What Are We Building?
Chapter 2	Bluetooth Low Energy in Plain Language
Chapter 3	How Linux Talks to Bluetooth: BlueZ and D-Bus
Chapter 4	A Map of the Project
Chapter 5	The Foundations: common.h, log.c, dbus_helper.c
Chapter 6	adapter.c — Talking to the Bluetooth Adapter
Chapter 7	device_manager.c — Remembering Devices We See
Chapter 8	signal_handler.c — Listening for BlueZ's Announcements
Chapter 9	advertisement.c — Making Ourselves Visible
Chapter 10	gatt_server.c — The Heart of the Project
Chapter 11	main.c — Putting It All Together
Chapter 12	Building, Running, and Testing With Your Phone
Chapter 13	Ideas for Extending the Project
Appendix A	Glossary of Terms

Introduction: What Are We Building?

Somewhere in the middle of this project's ten files, a small piece of software quietly does something remarkable: it turns an ordinary Linux computer into a **Bluetooth Low Energy device** that a phone can discover, connect to, read from, write to, and receive live updates from — the same way a fitness tracker or a smart bulb works.

This book walks through that project from the ground up. We will not assume you already know what GATT, D-Bus, or BlueZ mean. Each new idea is introduced before the code that uses it, and every source file gets its own chapter with diagrams showing exactly what happens, in what order, and why.

By the end, you should be able to open any of the ten files, and instead of seeing a wall of unfamiliar function calls, see a clear story: "*ah, this is registering an object on D-Bus so that BlueZ can find it.*"



In Bluetooth Low Energy, one side advertises (Peripheral) and one side listens & connects (Central).

Figure 1.1 — This project plays the Peripheral role: it advertises itself and waits for a phone (the Central) to connect.

What the finished project does

Concretely, once running, the program:

- Prints the name of your computer's Bluetooth adapter.
- Starts scanning the air for other nearby Bluetooth devices, and keeps a small table of everything it sees.
- Publishes one Bluetooth "service" with two "characteristics" — one you can read and write like a tiny shared notebook, and one that reports a simulated LED's on/off state and pushes live updates whenever it changes.
- Advertises itself over the air under the name `BLE-Framework` so that an app such as **nRF Connect** (a free phone app used to explore BLE devices) can find and connect to it.

Every one of those bullet points corresponds to one source file, and we will take them one at a time, in the order the code actually runs.

NOTE This project talks to BlueZ, the standard Bluetooth stack on Linux — so everything you learn here applies directly whether you run it on a desktop, a Raspberry Pi, or a small embedded board like the Luckfox Pico that a comment in the code mentions.

Bluetooth Low Energy in Plain Language

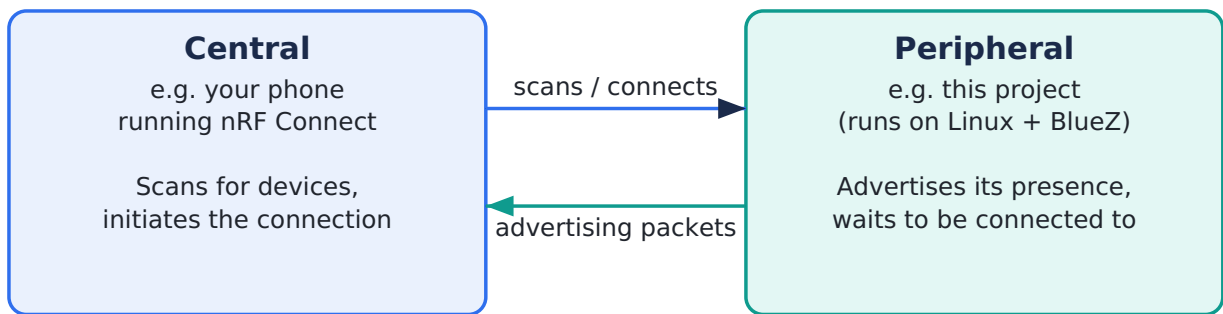
Before touching any code, we need four ideas. They show up in almost every function name in this project, so it's worth getting comfortable with them here.

2.1 Central and Peripheral

A Bluetooth Low Energy (BLE) connection always has two sides:

- **Peripheral** — the smaller, usually battery-powered device. It broadcasts ("advertises") short radio packets saying "I exist, here's my name" and then waits.
- **Central** — the device that listens for those packets and decides to connect. Your phone is almost always the Central.

This project's C program plays the **Peripheral** role. Your phone, running an app like nRF Connect, plays the **Central** role.



In Bluetooth Low Energy, one side advertises (Peripheral) and one side listens & connects (Central).

Figure 2.1 — Roles in a BLE connection.

2.2 GATT: Services, Characteristics, and UUIDs

Once connected, a Central doesn't just get one big blob of data from a Peripheral. Instead, the data is organized into a small tree, defined by a specification called **GATT** (Generic Attribute Profile):

- A **Service** is a logical grouping of related data — think of it as a folder. A real device usually has several (e.g. "Battery Service", "Heart Rate Service").
- A **Characteristic** is a single piece of data inside a service — think of it as a file inside that folder. Each characteristic has *flags* saying what you're allowed to do with it: `read`, `write`, `notify`, and others.
- A **Descriptor** is optional extra metadata about a characteristic (this project doesn't use any, but the diagram below shows where they would sit).

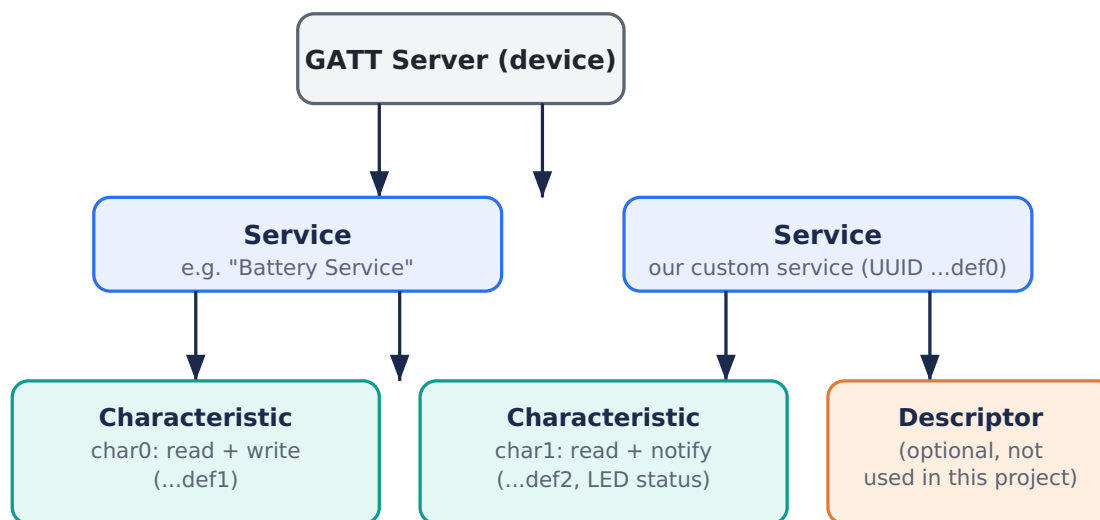


Figure 2.2 — The GATT tree: Device → Service → Characteristic → (optional) Descriptor.

Every service and characteristic is identified by a **UUID** — a 128-bit number written as a string like `12345678-1234-5678-1234-56789abcdef0`. Some UUIDs are officially standardized (battery level, heart rate...); this project defines its own three custom UUIDs, since it's a demo service rather than a standard one — you'll see them defined right at the top of `gatt_server.h`.

Two operations deserve special attention, because they behave differently from a normal function call:

- **Read / Write** are pulled by the Central: it explicitly asks "give me the current value" or "here is a new value, store it".
- **Notify** is pushed by the Peripheral: once a Central has subscribed (by calling `StartNotify`), the Peripheral can send new values on its own schedule, without being asked each time. This is exactly how the LED-status characteristic in this project works — it pushes a new value every time the simulated LED toggles.

2.3 Advertising

Before any of the above is possible, a Central has to find the Peripheral in the first place. That happens through **advertising**: the Peripheral periodically broadcasts a small packet containing things like a device name and a list of service UUIDs it offers, with no connection required. Scanning apps like nRF Connect show you a live list of every advertisement they hear.

NOTE Advertising and GATT are separate mechanisms. Advertising is how you get *found*; GATT is what happens *after* you're connected. This project has one file for each: `advertisement.c` and `gatt_server.c`.

How Linux Talks to Bluetooth: BlueZ and D-Bus

Nothing in this project talks to the Bluetooth radio directly. Instead, every single Bluetooth operation — starting a scan, registering a service, sending a notification — is done by sending a message across **D-Bus** to a background program called **bluetoothd**, which is part of the **BlueZ** project (BlueZ is simply the name of Linux's official Bluetooth stack).

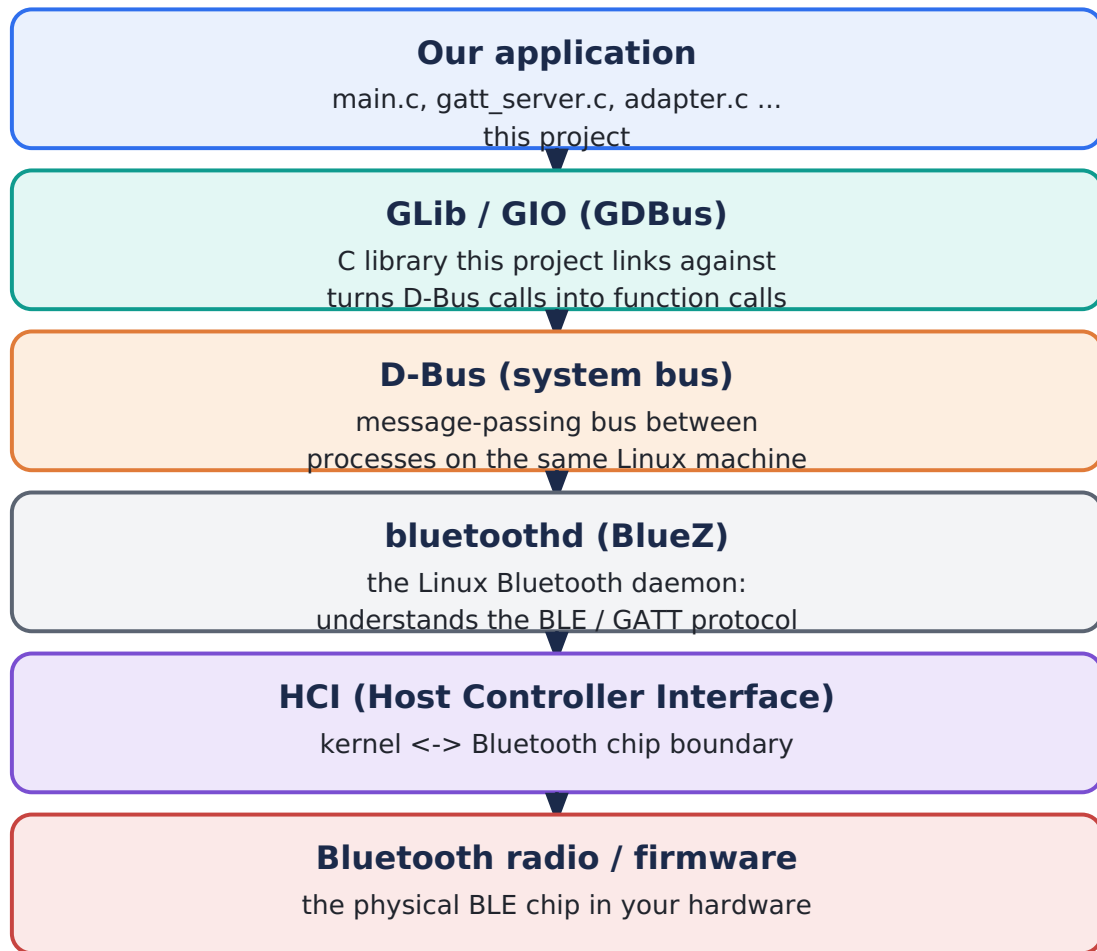


Figure 3.1 — Every layer this project's C code passes through before a signal reaches the actual radio.

3.1 What is D-Bus?

D-Bus is a message bus: a way for unrelated programs running on the same Linux machine to call each other's functions and send each other notifications, without knowing anything about how the other program is implemented. Two ideas from D-Bus appear constantly in this project's code:

- **Object paths** — a D-Bus service exposes "objects" at path-like names, e.g. `/org/bluez/hci0` is the object representing your Bluetooth adapter, and `/org/bluez/example/service0` (defined by *this* project) is the object representing our custom GATT service.
- **Interfaces** — each object supports one or more named interfaces, e.g. `org.bluez.Adapter1` or `org.bluez.GattCharacteristic1`. An interface is essentially a contract: "any object implementing me has these methods and these properties."

Programs interact with D-Bus objects in three ways, and you'll see all three in this codebase:

- **Method calls** — "please run this function and give me the result" (e.g. this project calling BlueZ's `StartDiscovery`, or BlueZ calling this project's `ReadValue`).
- **Properties** — named values an object exposes, readable via a generic `Get` / `GetAll` call.
- **Signals** — a one-way broadcast: "something happened, and I'm not waiting for a reply." BlueZ uses signals to tell us about newly discovered devices (`InterfacesAdded`) and property changes (`PropertiesChanged`).

3.2 Two D-Bus roles this project plays at once

This is the part that trips people up the most, so it's worth stating plainly: this program is simultaneously a D-Bus **client** and a D-Bus **server**.

- As a **client**, it calls methods that BlueZ exposes — e.g. `adapter_start_discovery()` calls BlueZ's `StartDiscovery` method, and subscribes to BlueZ's signals.
- As a **server**, it registers its own objects (the advertisement, the GATT service and its characteristics) on the bus, so that *BlueZ* can call methods on *them* — e.g. BlueZ calls our `ReadValue` when a phone asks to read a characteristic.

Whenever you see `g_dbus_connection_call()` in the code, we're acting as a client. Whenever you see `g_dbus_connection_register_object()`, we're acting as a server, publishing something for BlueZ to call into.

NOTE This project uses **GLib** and its D-Bus module **GDBus** (the functions starting with `g_` and types like `GVariant`) as a C library for talking to D-Bus — you never have to hand-craft raw D-Bus wire messages yourself.

3.3 GVariant: the "universal box" for D-Bus data

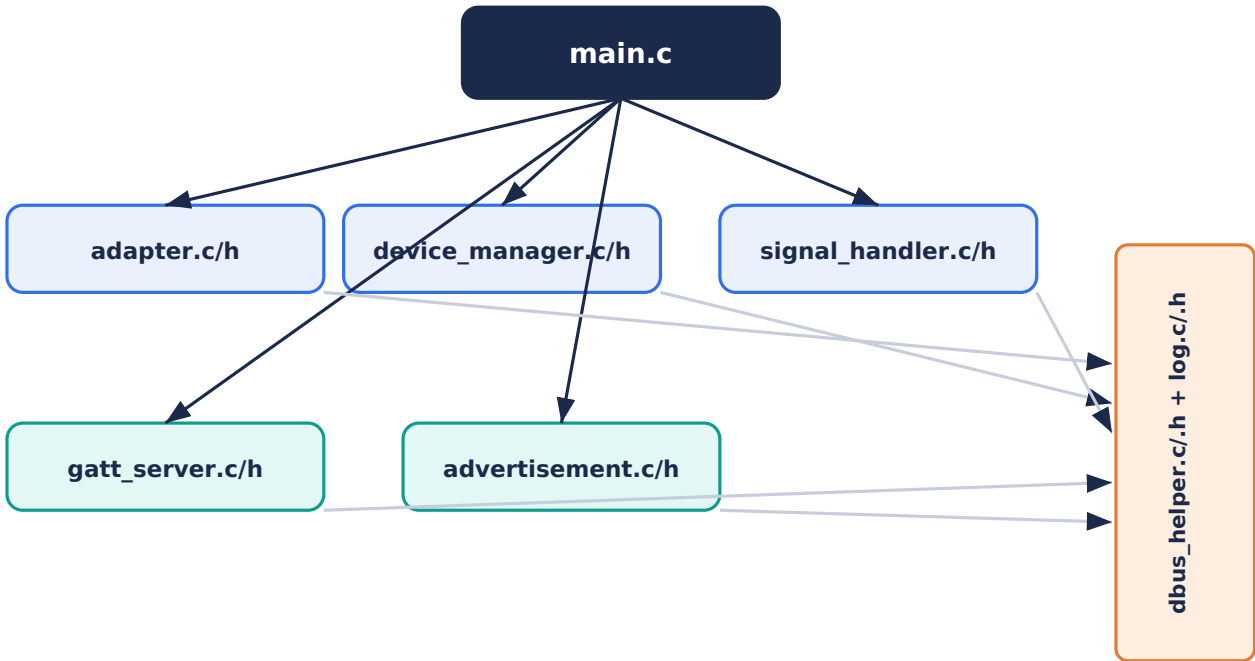
D-Bus messages carry strongly-typed data, and GLib represents any piece of that data — a string, an integer, an array of bytes, or a whole nested dictionary — using a single type: `GVariant`. Every `GVariant` has a **type string** that describes its shape, for example:

Type string	Meaning
s	a string
b	a boolean
o	an object path
ay	an array of bytes ("array of y", where y = byte) — used for characteristic values
as	an array of strings
a{sv}	a dictionary from string keys to "variant" values — used everywhere for property bags
a{oa{sa{sv}}}	the big one: object-path → interface-name → property-dictionary. This is exactly the shape BlueZ expects from <code>GetManagedObjects()</code> .

You don't need to memorize these — you'll see each one explained again the moment it appears in real code. The important habit is: whenever you see a quoted type string like `"(oa{sv})"`, read it as a description of the data's shape, not as magic.

A Map of the Project

With those concepts in hand, here is how the ten source files divide the work. Keep this page as a reference — every later chapter zooms into exactly one box.



Every module talks to BlueZ only through `dbus_helper.c`, and logs status through `log.c`.

Figure 4.1 — `main.c` starts everything; two shared helper files sit underneath all of it.

File	Responsibility
<code>main.c</code>	Program entry point. Calls each module's <code>_init()</code> in order, then runs the GLib main loop forever.
<code>common.h</code>	One shared header pulling in standard C and GLib/GIO headers, included by almost every file.
<code>log.c / log.h</code>	Two tiny functions, <code>log_info</code> and <code>log_error</code> , used everywhere instead of raw <code>printf</code> .
<code>dbus_helper.c / .h</code>	Opens the one shared connection to the D-Bus system bus, and a generic helper that pretty-prints any property dictionary for logging.
<code>adapter.c / .h</code>	Talks to the local Bluetooth adapter object: prints its name, starts/stops scanning.
<code>device_manager.c / .h</code>	A small fixed-size table that remembers every device we've heard about while scanning.
<code>signal_handler.c / .h</code>	Subscribes to BlueZ's D-Bus signals and updates the device table whenever BlueZ reports something new.
<code>advertisement.c / .h</code>	Publishes our own LE advertisement object and registers it with BlueZ so we become discoverable.
<code>gatt_server.c / .h</code>	Publishes our custom GATT service and its two characteristics, and simulates an LED that toggles every 3 seconds.

4.1 Two independent stories happening at once

- It helps to notice that this program is really doing two unrelated jobs concurrently, tied together only by the shared main loop:
- Being a Central for a moment** — `adapter.c` and `signal_handler.c` together scan for and log *other* nearby Bluetooth devices (chapters 6–8).
 - Being a Peripheral** — `advertisement.c` and `gatt_server.c` together make *this* machine discoverable and connectable by a phone (chapters 9–10).

Nothing stops you from deleting one half and keeping the other — they don't depend on each other. The project just runs both to demonstrate each side of BLE in one place.

The Foundations: common.h, log.c, dbus_helper.c

Three small files do the unglamorous but essential work every other module leans on. We cover all three together because none of them is more than a page of real logic.

5.1 common.h — one header to rule them all

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#include <gio/gio.h>

#endif
```

This is a convenience header: instead of every file separately including the standard C headers plus `<gio/gio.h>` (GLib's I/O and D-Bus module), they just write `#include "common.h"` once. `gio.h` is what brings in types like `GDBusConnection`, `GVariant`, and `Gboolean` that you'll see throughout the project.

5.2 log.c — a two-function logging library

log.c

```
void log_info(const char *fmt, ...)
{
    va_list args;

    printf("[INFO] ");

    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);

    printf("\n");
}

void log_error(const char *fmt, ...)
{
    va_list args;

    printf("[ERROR] ");

    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);

    printf("\n");
}
```

If you haven't seen the `va_list` / `va_start` / `vprintf` trio before: this is standard C's mechanism for writing a function that, like `printf` itself, accepts a format string followed by a variable number of extra arguments. `log_info` and `log_error` are simply `printf` with a `[INFO]` or `[ERROR]` tag stuck on the front and a newline on the end — used consistently everywhere instead of raw `printf`, so every line of output is easy to scan and filter (e.g. with `grep ERROR`).

TRY IT Run the finished program with `./ble_app 2>&1 | grep ERROR` to instantly see only the lines that indicate something went wrong.

5.3 dbus_helper.c — the one D-Bus connection everyone shares

dbus_helper.c (excerpt)

```
GDBusConnection *dbus_get_system_connection(void)
{
    GError *error = NULL;
    GDBusConnection *conn;

    conn = g_bus_get_sync(
        G_BUS_TYPE_SYSTEM,
        NULL,
        &error);

    if(error)
    {
        log_error("%s", error->message);
        g_error_free(error);
        return NULL;
    }

    log_info("Connected to System D-Bus");

    return conn;
}
```

`g_bus_get_sync(G_BUS_TYPE_SYSTEM, ...)` connects to the **system bus** — as opposed to a per-user "session bus" — because BlueZ's daemon (`bluetoothd`) registers itself on the system bus. Every module in this project that needs to talk to BlueZ (the adapter, the signal handler, the GATT server, the advertiser) calls this same function to get a connection, rather than opening several separate connections.

The `GError **error` pattern you see here is GLib's standard way of reporting failures: you pass in a pointer to an error pointer, initialized to `NULL`; if the call fails, GLib allocates a `GError` and points your variable at it, which you must check and then free with `g_error_free()`. You will see this exact pattern repeated in nearly every function in this project that performs a D-Bus call.

5.4 dbus_print_properties — turning "any GVariant" into readable text

The second function in this file, `dbus_print_properties()`, walks a `a{sv}` dictionary (remember that type string from Chapter 3: string keys, variant values) and logs each key/value pair. Its real complexity lives in a helper, `dbus_variant_to_string()`, which recursively converts *any* GVariant — string, boolean, byte, integer of various widths, byte-string, or even an array of other variants — into a printable string. This is what lets the signal handler in Chapter 8 dump a device's full property list without needing a separate case for every possible field BlueZ might send.

NOTE You do not need to read every branch of `dbus_variant_to_string()` line by line — it's a dispatcher: "if it's this GVariant type, format it this way." Recognizing the pattern matters more than memorizing each branch.

adapter.c — Talking to the Bluetooth Adapter

Your Bluetooth chip is represented on D-Bus as an object — typically `/org/bluez/hci0` for the first adapter — implementing the interface `org.bluez.Adapter1`. This file contains three small functions, each making exactly one D-Bus call to that object.

6.1 Reading a property: `adapter_print_name()`

adapter.c — `adapter_print_name()` (excerpt)

```
result =
    g_dbus_connection_call_sync(
        conn,
        "org.bluez",
        "/org/bluez/hci0",
        "org.freedesktop.DBus.Properties",
        "Get",
        g_variant_new("(ss)",
            "org.bluez.Adapter1",
            "Name"),
        G_VARIANT_TYPE("(v)"),
        G_DBUS_CALL_FLAGS_NONE,
        -1,
        NULL,
        &error);

g_variant_get(result, "(v)", &value);
g_variant_get(value, "s", &name);
log_info("Bluetooth Adapter : %s", name);
```

Notice this doesn't call a method named "GetName" on the adapter. Properties in D-Bus are read through a *generic* interface, `org.freedesktop.DBus.Properties`, which every object automatically supports. Its `Get` method takes two strings — the interface name and the property name — and returns the value wrapped in a variant, hence the type string `"(v)"`. The code above unwraps it in two steps: first `g_variant_get(result, "(v)", &value)` pulls the variant out of the reply tuple, then a second `g_variant_get(value, "s", &name)` pulls the actual string out of that variant.

This same two-step unwrapping — "get the variant, then get the value inside the variant" — is worth remembering, because it reappears constantly whenever this project touches BlueZ properties.

6.2 Calling a method: `adapter_start_discovery()` and `adapter_stop_discovery()`

adapter.c — `adapter_start_discovery()` (excerpt)

```
g_dbus_connection_call_sync(
    conn,
    "org.bluez",
    "/org/bluez/hci0",
    "org.bluez.Adapter1",
    "StartDiscovery",
    NULL,
    NULL,
    G_DBUS_CALL_FLAGS_NONE,
    -1,
    NULL,
    &error);
```

This is a plain method call, this time on `org.bluez.Adapter1` directly: "call `StartDiscovery`, no arguments, and I don't care about a return value." `adapter_stop_discovery()` is identical except for calling `StopDiscovery`. Both use `g_dbus_connection_call_sync` — the *synchronous* variant, which blocks the program until BlueZ replies. That's a reasonable choice here since these calls happen once at startup and return almost immediately; you'll see the *asynchronous* variant, `g_dbus_connection_call()`, used instead in the GATT and advertisement chapters, where the call matters more and blocking briefly would be wasteful.

NOTE Starting discovery does not, by itself, add anything to our device table — it only tells BlueZ to start listening for advertisements. The device table is filled in separately by `signal_handler.c`, covered in Chapter 8, which reacts to BlueZ's announcements.

device_manager.c — Remembering Devices We See

Once discovery is running, BlueZ will tell us about every device it hears, potentially dozens of times each (once when first seen, then again every time a property like RSSI updates). This file's only job is to keep a simple, fixed-size record of what we currently know about each one.

BLE_DEVICE (one entry per discovered device, device_table[128])

used	gboolean	slot occupied?
object_path	char[128]	D-Bus object path, e.g. /org/bluez/hci0/dev_AA_BB_...
address	char[32]	Bluetooth MAC address
name / alias	char[64]	advertised / user-friendly name
address_type	char[16]	public or random
rssi	int16_t	signal strength (more negative = weaker)
connected / paired / trusted	gboolean	connection state flags
services_resolved	gboolean	GATT services discovered yet?

Figure 7.1 — Every field BlueZ has ever told us about one discovered device.

The whole table lives as one array, sized generously up front rather than growing dynamically:

```
device_manager.c (excerpt)

#define MAX_BLE_DEVICES      128

static BLE_DEVICE device_table[MAX_BLE_DEVICES];
```

This is a deliberate, simple design choice: no linked lists, no dynamic allocation, no risk of a memory leak — at the cost of a hard ceiling of 128 simultaneously-remembered devices, which is far more than you'll typically see in one room.

7.1 The three operations

`device_find_by_path()` does a straightforward linear scan of the table, comparing D-Bus object paths, and returns a pointer to the matching slot or `NULL`:

```
device_manager.c — device_find_by_path()

BLE_DEVICE *device_find_by_path(const char *path)
{
    for(int i=0; i<MAX_BLE_DEVICES; i++)
    {
        if(device_table[i].used && strcmp(device_table[i].object_path, path) == 0)
        {
            return &device_table[i];
        }
    }
    return NULL;
}
```

`device_add()` is careful to be idempotent: it first checks whether the device is already known (via the function above) and returns the existing entry if so, only claiming a fresh, unused slot when the device is genuinely new. That matters because, as you'll see in the next chapter, BlueZ's `InterfacesAdded` signal fires once *per interface* on a device, not once per device — so `device_add()` for the same device path can legitimately be called several times in quick succession.

Finally, `device_print_all()` is a simple debug dump, looping over every used slot and logging the address, name, RSSI, and connection state — handy for a quick sanity check that scanning is actually working.

TRY IT Add a call to `device_print_all()` on a timer (using `g_timeout_add_seconds()`, the same function the GATT server uses to blink its simulated LED) to get a live, periodically-refreshed list of everything nearby, instead of a wall of individual log lines.

signal_handler.c — Listening for BlueZ's Announcements

This is where the "Central" half of the project actually comes alive. Starting discovery (Chapter 6) only asks BlueZ to start listening; it's this file's job to react when BlueZ finds something and tell `device_manager.c` about it.

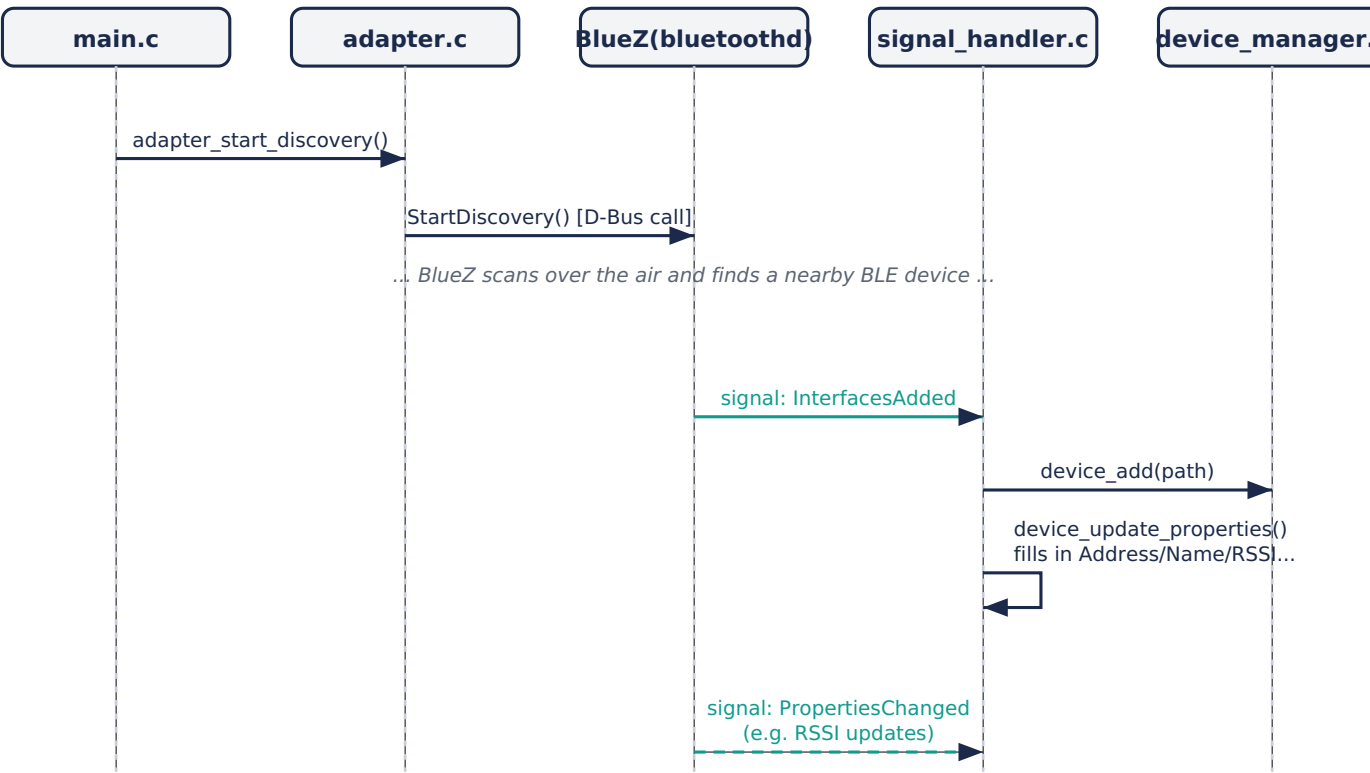


Figure 8.1 — From starting discovery to a fully populated device table entry.

8.1 Subscribing to signals

`signal_handler_init()` registers two callback functions with GDBus, each tied to one specific D-Bus signal:

```
signal_handler.c — signal_handler_init() (excerpt)

g_dbus_connection_signal_subscribe(
    conn,
    "org.bluez",
    "org.freedesktop.DBus.ObjectManager",
    "InterfacesAdded",
    NULL, NULL,
    G_DBUS_SIGNAL_FLAGS_NONE,
    interfaces_added_callback,
    NULL, NULL);

g_dbus_connection_signal_subscribe(
    conn,
    "org.bluez",
    "org.freedesktop.DBus.Properties",
    "PropertiesChanged",
    NULL, NULL,
    G_DBUS_SIGNAL_FLAGS_NONE,
    properties_changed_callback,
    NULL, NULL);
```

From this point on, GDBus takes care of everything: whenever BlueZ (sender `"org.bluez"`) emits a matching signal, our callback function will simply be invoked with the signal's payload — we never poll for anything.

8.2 InterfacesAdded — a new device appears

`org.freedesktop.DBus.ObjectManager` is another generic, standard D-Bus interface (much like `Properties` from Chapter 6). BlueZ implements it to announce whenever a new object shows up under its management — which happens the moment it hears an advertisement from a device it hasn't reported yet:

```

g_variant_get(parameters, "(o@a{sa{sv}})", &device_path, &interfaces);

BLE_DEVICE *device;
device = device_add(device_path);

GVariantIter iter;
const gchar *iface_name;
GVariant *properties;

g_variant_iter_init(&iter, interfaces);

while (g_variant_iter_next(&iter, "{s@a{sv}}", &iface_name, &properties))
{
    log_info("Interface : %s", iface_name);
    device_update_properties(device, properties);
    g_variant_unref(properties);
}

```

Reading the type string `"(o@a{sa{sv}})"` the way Chapter 3 suggested: this signal's payload is a tuple of an object path (`o`) — the new device's D-Bus path — followed by a dictionary from interface name to property dictionary (`a{sa{sv}}`). A single device can implement several interfaces at once (most relevantly `org.bluez.Device1`), so the code loops over all of them, calling the file's other key function, `device_update_properties()`, for each one.

8.3 device_update_properties — filling in the blanks

This function (defined at the top of the file, since both callbacks use it) is a long but simple `if / else if` chain: for each known property name — `Address`, `Name`, `Alias`, `RSSI`, `Connected`, `Paired`, `Trusted`, `ServicesResolved` — it pulls the value out of the `GVariant` with the matching accessor (`g_variant_get_string`, `g_variant_get_int16`, `g_variant_get_boolean` ...) and copies it into the corresponding `BLE_DEVICE` field from Chapter 7's struct. Any property BlueZ sends that this project doesn't recognize is silently skipped — which is exactly why `device_update_properties()` is a growing, not a fixed, list: add another `else if` branch any time you want to track one more field.

8.4 PropertiesChanged — an existing device changes

The second callback, `properties_changed_callback()`, fires continuously after the initial discovery — for example every time a device's RSSI is re-measured. It deliberately filters to only care about `org.bluez.Device1` changes (ignoring, say, changes on the adapter object itself), then simply logs the changed properties using `dbus_print_properties()` from Chapter 5, rather than updating the table — a reasonable simplification for a demo project, and a natural first improvement to make it call `device_update_properties()` too.

NOTE Look closely and you'll notice `properties_changed_callback()` never looks up the device via `device_find_by_path()` before logging — it prints the raw changed properties regardless of whether that device is already in our table. That's fine for logging, but worth keeping in mind if you extend this callback to update state.

advertisement.c — Making Ourselves Visible

Everything so far made us a Central, listening for others. This chapter flips the role: it's what makes a phone running nRF Connect able to find *this computer* in a scan list at all.

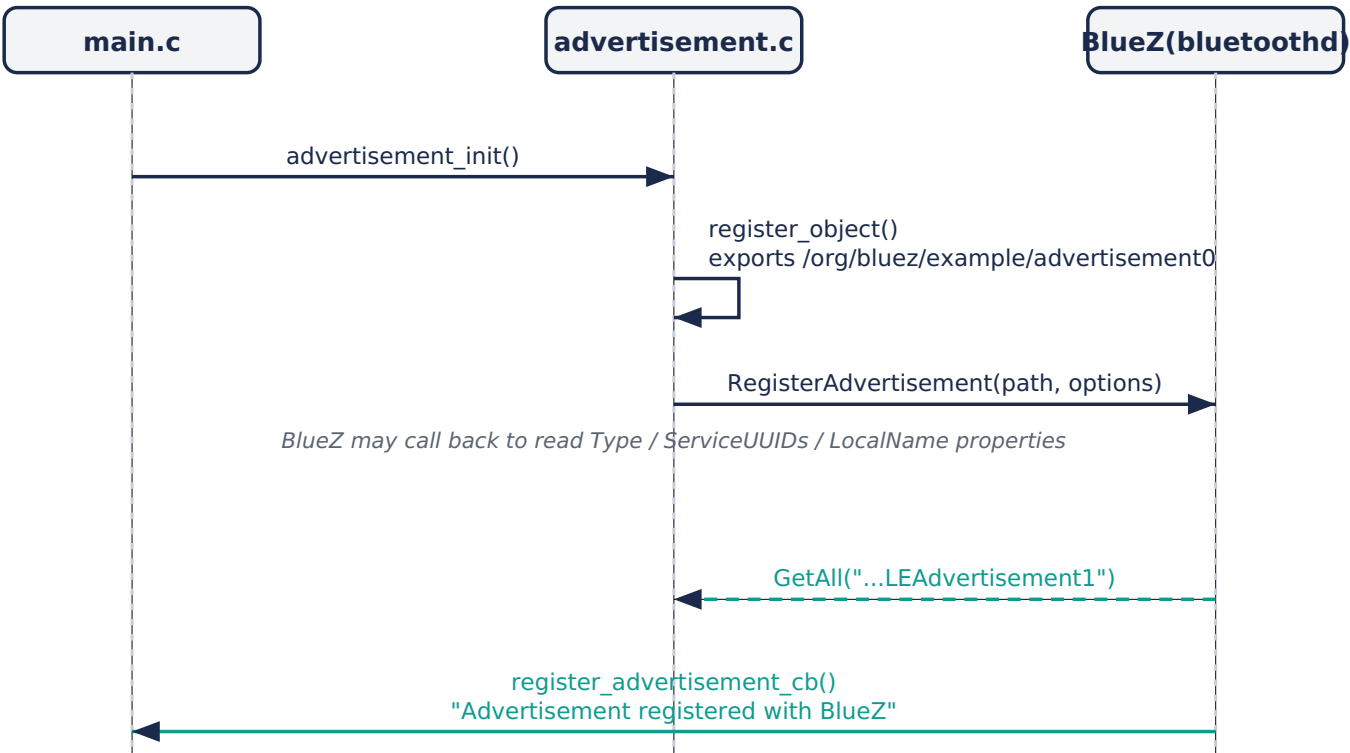


Figure 9.1 — Publishing our own advertisement object and registering it with BlueZ.

9.1 Describing ourselves with an introspection XML string

To register any object on D-Bus, GDBus needs to know its "shape" — which interface it implements, and what properties and methods that interface has. This project describes that shape as a small XML string, parsed at runtime:

advertisement.c — advertisement_xml

```
static const gchar *advertisement_xml =
    "<node>"
    "  <interface name='org.bluez.LEAdvertisement1'>"
    "    <property type='s' name='Type' access='read' />"
    "    <property type='as' name='ServiceUUIDs' access='read' />"
    "    <property type='s' name='LocalName' access='read' />"
    "    <property type='b' name='IncludeTxPower' access='read' />"
    "    <method name='Release' />"
    "  </interface>"
    "</node>";
```

This says: "I implement `org.bluez.LEAdvertisement1`, with four read-only properties and one method." `org.bluez.LEAdvertisement1` is BlueZ's defined interface for anything that wants to be advertised — this project isn't inventing it, just implementing it.

9.2 Answering BlueZ's questions: the vtable

Once registered, BlueZ (or GDBus internally) may call back into our object to read one of those properties, or invoke `Release`. GDBus dispatches those callbacks through a `GDBusInterfaceVTable` — a small struct of function pointers:

advertisement.c — advertisement_vtable

```
static const GDBusInterfaceVTable advertisement_vtable = {
    .method_call = advertisement_method_call,
    .get_property = advertisement_get_property,
    .set_property = NULL
};
```

`advertisement_get_property()` is a simple dispatcher on property name, building each value on the fly with the matching `g_variant_new_*` constructor — e.g. `g_variant_new_strv(uuids, -1)` for `ServiceUUIDs`, where `uuids` is a tiny array containing just this project's `GATT_SERVICE_UUID` (defined in `gatt_server.h` — the advertisement deliberately advertises the same service the GATT server will expose, so a scanning app can tell in advance what this device offers). `advertisement_method_call()` handles the one method, `Release`, which BlueZ calls if it ever needs to un-advertise us — the handler just logs it and returns an empty reply.

9.3 Registering the object, then registering it *with BlueZ*

There are two separate registration steps here, and it's easy to conflate them:

1. `g_dbus_connection_register_object()` — a purely local step. It tells *our own* GDBus connection: "if anyone calls a method on `/org/bluez/example/advertisement0`, route it to `advertisement_vtable`."
2. `g_dbus_connection_call(..., "RegisterAdvertisement", ...)` — a remote step. It tells *BlueZ*: "please start advertising the object at that path."

Only after both steps have happened does BlueZ actually start reading our properties and broadcasting packets over the radio. Notice this call uses the *asynchronous* `g_dbus_connection_call()` (unlike Chapter 6's adapter calls) with a callback, `register_advertisement_cb()`, which simply logs success or a descriptive failure — including a friendly hint that the adapter might need to be powered on first.

NOTE If `RegisterAdvertisement` fails, the most common cause (as the code's own log message hints) is that the adapter is powered off. From a terminal: `bluetoothctl power on`.

gatt_server.c — The Heart of the Project

This is the longest file in the project, and for good reason: it single-handedly builds an entire small GATT tree (Chapter 2) — one service and two characteristics — and registers all of it with BlueZ. We'll take it one object at a time, in the same order the file defines them.

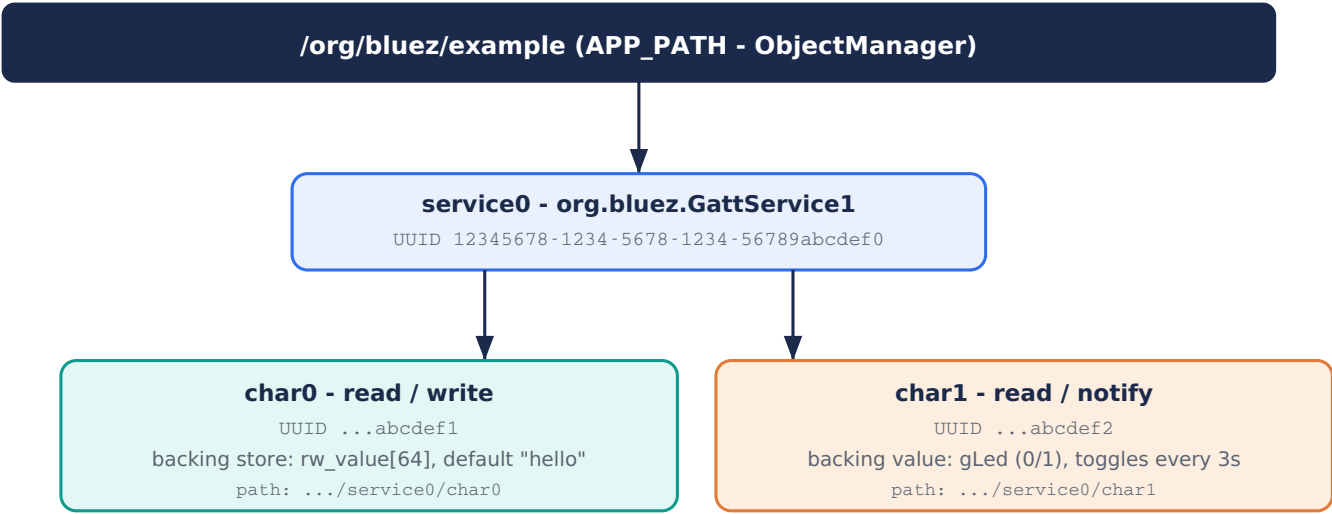


Figure 10.1 — The exact object tree this file publishes on D-Bus, with real UUIDs and paths.

10.1 Four objects, four D-Bus paths

Right at the top, four path constants lay out the whole tree as plain strings:

gatt_server.c — path constants

```
#define APP_PATH "/org/bluez/example"
#define SERVICE_PATH APP_PATH "/service0"
#define CHAR_RW_PATH SERVICE_PATH "/char0"
#define CHAR_NOTIFY_PATH SERVICE_PATH "/char1"
```

D-Bus object paths are hierarchical purely by convention — nesting `char0` under `service0` under `example` has no special technical meaning to D-Bus itself, but it mirrors the logical GATT hierarchy from Chapter 2 and makes the tree easy to read in a debugging tool. Four objects will be registered at these four paths:

- `APP_PATH` — implements `org.freedesktop.DBus.ObjectManager`, the "front door" BlueZ uses to discover everything else.
- `SERVICE_PATH` — implements `org.bluez.GattService1`.
- `CHAR_RW_PATH` — implements `org.bluez.GattCharacteristic1` (read/write).
- `CHAR_NOTIFY_PATH` — implements `org.bluez.GattCharacteristic1` (read/notify).

10.2 The registration flow, end to end

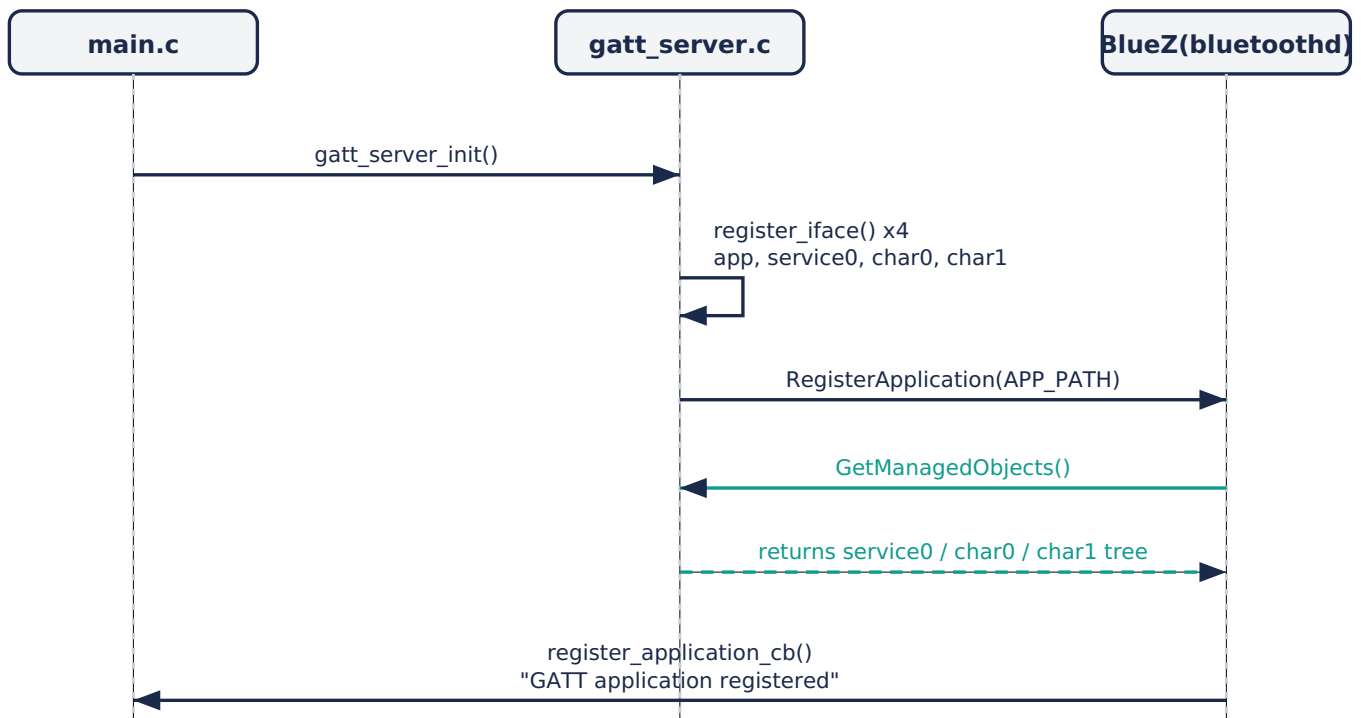


Figure 10.2 — What happens, in order, from `gatt_server_init()` to BlueZ fully understanding our service.

All four objects are registered through one small shared helper, so it's worth reading this first:

gatt_server.c — `register_iface()` (excerpt)

```

static guint register_iface(GDBusConnection *conn, const gchar *path,
                           const gchar *xml, const GDBusInterfaceVTable *vtable)
{
    GError *error = NULL;
    GDBusNodeInfo *node = g_dbus_node_info_new_for_xml(xml, &error);
    ...
    guint id = g_dbus_connection_register_object(
        conn, path, node->interfaces[0], vtable, NULL, NULL, &error);
    ...
    return id;
}

```

This mirrors exactly the two-part registration pattern from Chapter 9 (parse an introspection XML string, then register the object with a vtable), just factored into a reusable function since this file needs it four times. Then `gatt_server_init()` calls it once per object, followed by the one remote call that actually tells BlueZ about all of it:

gatt_server.c — `gatt_server_init()` (excerpt)

```

register_iface(gatt_conn, APP_PATH, object_manager_xml, &object_manager_vtable);
register_iface(gatt_conn, SERVICE_PATH, gatt_service_xml, &gatt_service_vtable);
register_iface(gatt_conn, CHAR_RW_PATH, char_rw_xml, &char_rw_vtable);
register_iface(gatt_conn, CHAR_NOTIFY_PATH, char_notify_xml, &char_notify_vtable);

g_dbus_connection_call(
    gatt_conn, "org.bluez", ADAPTER_PATH,
    "org.bluez.GattManager1", "RegisterApplication",
    g_variant_new("(oa{sv})", APP_PATH, &empty_options),
    NULL, G_DBUS_CALL_FLAGS_NONE, -1, NULL,
    register_application_cb, NULL);

```

Note the order: everything is registered *locally first*, and only then does `RegisterApplication` tell BlueZ where to look (`APP_PATH`). This matters because of what happens next.

10.3 GetManagedObjects — describing the whole tree in one shot

Immediately after `RegisterApplication` succeeds, BlueZ calls back into *our* `APP_PATH` object asking for `GetManagedObjects()` — this is how it learns about `service0`, `char0`, and `char1` without us having to separately announce each one. The handler simply builds and returns the answer:

```
static GVariant *build_managed_objects(void)
{
    GVariantBuilder objects;
    g_variant_builder_init(&objects, G_VARIANT_TYPE("a{oa{sa{sv}}}")");

    /* ---- service0 : org.bluez.GattService1 ---- */
    {
        GVariantBuilder props;
        g_variant_builder_init(&props, G_VARIANT_TYPE("a{sv}"));
        g_variant_builder_add(&props, "{sv}", "UUID", g_variant_new_string(GATT_SERVICE_UUID));
        g_variant_builder_add(&props, "{sv}", "Primary", g_variant_new_boolean(TRUE));

        GVariantBuilder ifaces;
        g_variant_builder_init(&ifaces, G_VARIANT_TYPE("a{sa{sv}}"));
        g_variant_builder_add(&ifaces, "{sa{sv}}", "org.bluez.GattService1", &props);

        g_variant_builder_add(&objects, "a{oa{sa{sv}}}", SERVICE_PATH, &ifaces);
    }
    /* ...char0 and char1 built the same way... */

    return g_variant_new("a{oa{sa{sv}}}", &objects);
}
```

This is the type string `a{oa{sa{sv}}}` from Chapter 3, built by hand: for each object path, a dictionary of interface-name to property-dictionary. Reading the nesting from the inside out: `props` holds one object's properties, wrapped into `ifaces` under an interface name, wrapped into `objects` under an object path. `GVariantBuilder` is GLib's tool for assembling a `GVariant` piece by piece like this, since `GVariants` themselves are immutable once built.

NOTE Look carefully and you'll notice the two characteristics' `Flags` property is what actually tells a BLE client what it's allowed to do — `["read","write"]` for `char0`, `["read","notify"]` for `char1` — not anything in the method-call handlers themselves. BlueZ enforces those flags before ever calling into your `WriteValue` or `StartNotify` handlers.

10.4 char0 — a tiny read/write notebook

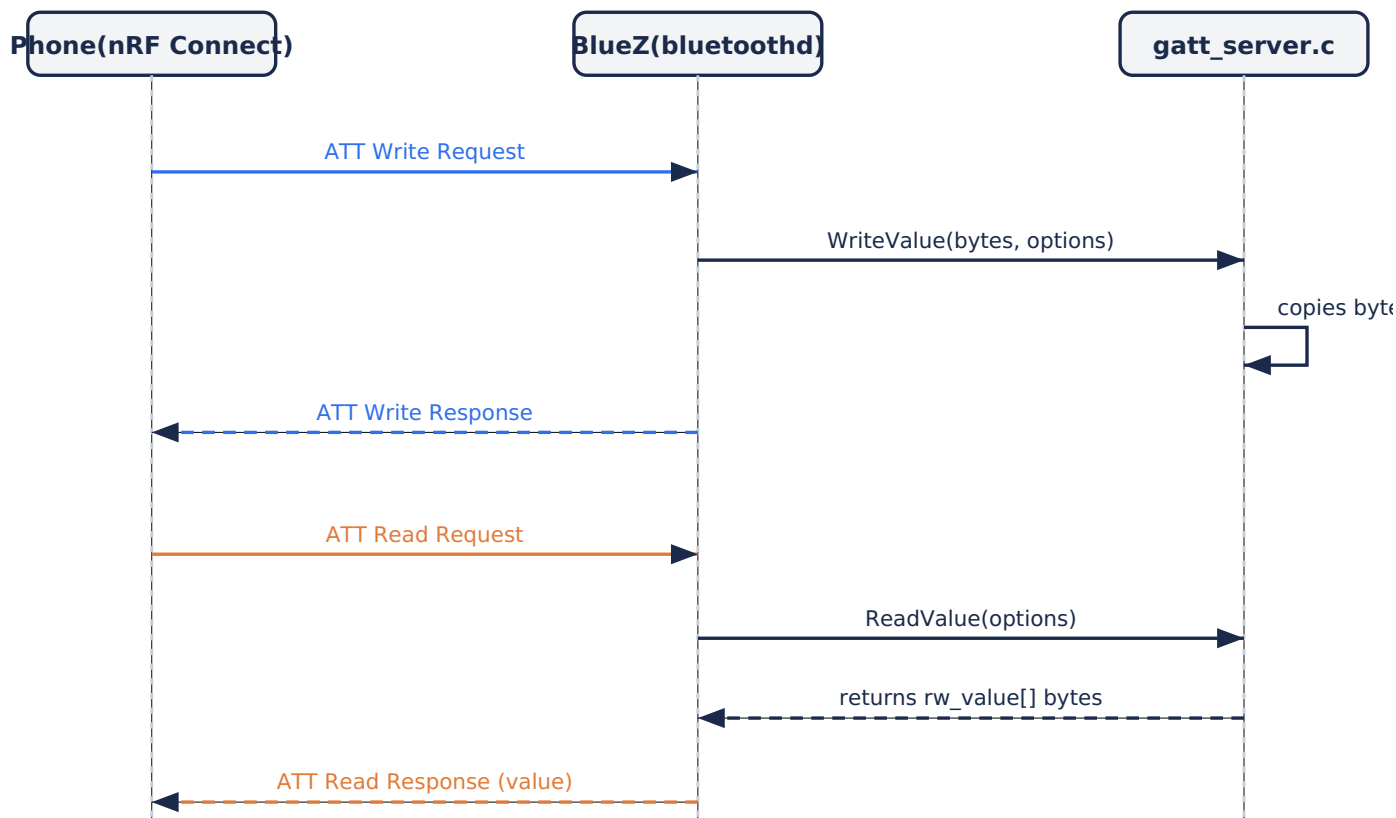


Figure 10.3 — What happens on the wire when a phone reads or writes `char0`.

`char0`'s value lives in a plain static byte buffer:

gatt_server.c — char0 backing storage

```
static quint8 rw_value[64] = "hello";
static gsize  rw_value_len = 5;
```

`WriteValue` copies whatever bytes the phone sent into that buffer:

gatt_server.c — char_rw_method_call(), WriteValue branch (excerpt)

```
g_variant_get(parameters, "(ay@a{sv})", &iter, &options);

while (i < sizeof(rw_value) - 1 && g_variant_iter_next(iter, "y", &byte))
    rw_value[i++] = byte;
rw_value_len = i;
rw_value[i] = '\0';
```

The parameters type string, `(ay@a{sv})`, is a tuple of the value being written (`ay`, an array of bytes) and an options dictionary (`@a{sv}`) — this project doesn't use any options, but BlueZ always sends the dictionary). The loop copies each byte in one at a time using a `GVariantIter`, stopping safely before overflowing the 64-byte buffer, and adds a trailing `'\0'` so the buffer can also be logged as a normal C string if the phone happened to write plain text.

`ReadValue` does the reverse — building a fresh `ay` array from the current buffer contents and returning it. Because both handlers touch the same `rw_value` variable, this characteristic behaves exactly like a shared notebook: whatever the last write said is what the next read returns.

10.5 char1 — a simulated LED, and how "notify" really works

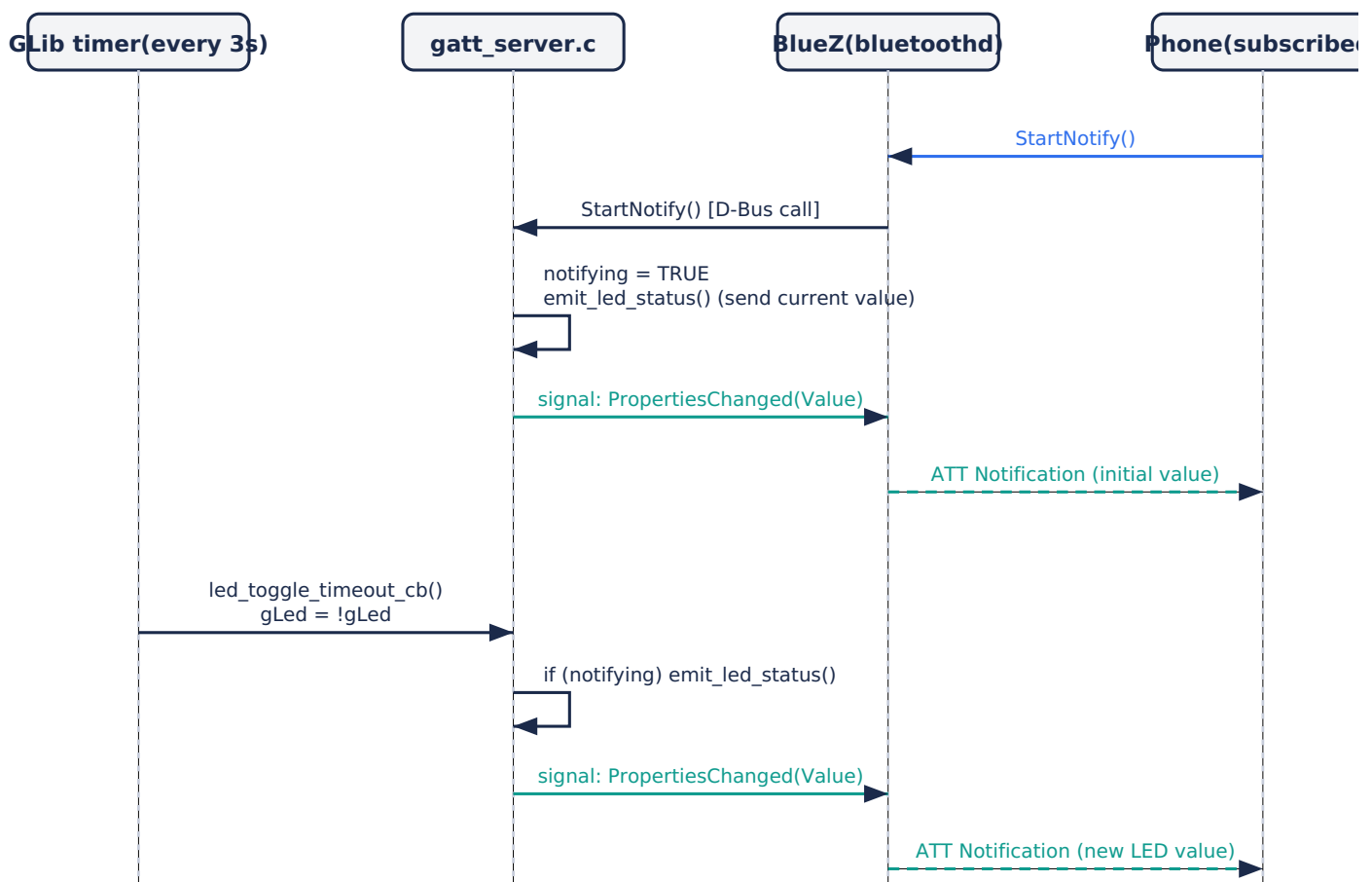


Figure 10.4 — The full life cycle: subscribing, then receiving a push update on every LED toggle.

This is the most interesting characteristic in the project, because it demonstrates the *push* side of GATT introduced in Chapter 2. Three pieces work together:

a) A timer that runs regardless of anyone watching

gatt_server.c — led_toggle_timeout_cb() (excerpt)

```
static gboolean led_toggle_timeout_cb(gpointer user_data)
{
    gLed = !gLed;
    log_info("LED status: %s", gLed ? "ON" : "OFF");

    /* On real hardware, replace this with the actual GPIO write. */

    if (notifying)
        emit_led_status();

    return G_SOURCE_CONTINUE;
}

/* registered in gatt_server_init(): */
g_timeout_add_seconds(3, led_toggle_timeout_cb, NULL);
```

`g_timeout_add_seconds(3, ...)` tells GLib's main loop (the same loop started in `main.c` and covered in Chapter 11) to call this function every three seconds for the rest of the program's life. Returning `G_SOURCE_CONTINUE` is what keeps it repeating — returning `G_SOURCE_REMOVE` instead would cancel it after one run. Crucially, the LED keeps blinking whether or not a phone is connected, exactly as a real embedded blink loop would; the comment even points to where you'd swap in a real `gpio_set_value()` call on actual hardware such as the Luckfox Pico Plus mentioned in the header comments.

b) Subscribing: StartNotify / StopNotify

gatt_server.c — char_notify_method_call(), StartNotify branch

```
else if (g_strcmp0(method_name, "StartNotify") == 0)
{
    if (!notifying)
    {
        notifying = TRUE;
        emit_led_status(); /* push the current value right away */
    }
    g_dbus_method_invocation_return_value(invocation, NULL);
}
```

A phone that wants live updates calls `StartNotify` once. This project tracks that with a single `static gboolean notifying` flag — a simplification worth noticing: it doesn't track *which* client subscribed, just whether *anyone* currently has. That's fine for a demo with one connected phone at a time, but is exactly the kind of detail you'd revisit for a production multi-client server.

c) Pushing: emit_led_status()

gatt_server.c — emit_led_status()

```
static void emit_led_status(void)
{
    GVariantBuilder array;
    g_variant_builder_init(&array, G_VARIANT_TYPE("ay"));
    g_variant_builder_add(&array, "y", gLed);

    GVariantBuilder changed;
    g_variant_builder_init(&changed, G_VARIANT_TYPE("a{sv}"));
    g_variant_builder_add(&changed, "{sv}", "Value", g_variant_new("ay", &array));

    GVariantBuilder invalidated;
    g_variant_builder_init(&invalidated, G_VARIANT_TYPE("as"));

    g_dbus_connection_emit_signal(
        gatt_conn, NULL, CHAR_NOTIFY_PATH,
        "org.freedesktop.DBus.Properties", "PropertiesChanged",
        g_variant_new("{sa{sv}as}", "org.bluez.GattCharacteristic1", &changed, &invalidated),
        NULL);
}
```

Here is the detail that makes push notifications work, and it's subtle enough to call out explicitly: **this project never calls a function named "Notify" or "SendNotification."** Instead, it emits a completely generic `org.freedesktop.DBus.Properties.PropertiesChanged` signal — the very same kind of signal Chapter 8 subscribed to for *other* devices — announcing that our own `Value` property changed. BlueZ is watching for exactly this signal on GATT characteristic objects it manages, and it is BlueZ, not this code, that translates it into an actual low-level BLE ATT notification sent over the air to every subscribed client. Our C code never touches the radio, or even the concept of "which client is connected" — it just states a fact ("my Value property changed") and trusts BlueZ to act on it.

TRY IT This is a genuinely reusable pattern: any time you want a characteristic to push updates, give it a `notify` flag, and call `emit_led_status()` - style code (renamed for your data) any time the underlying value changes — a sensor reading, a counter, anything.

10.6 A quick before/after: this file compared to the D-Bus object model

D-Bus concept (Chapter 3)	Where it appears in gatt_server.c
Registering a server-side object	<code>register_iface()</code> , called 4 times
A generic interface every object can support	<code>org.freedesktop.DBus.ObjectManager</code> at APP_PATH
A method call from BlueZ to us	<code>ReadValue</code> , <code>WriteValue</code> , <code>StartNotify</code> , <code>StopNotify</code>
A method call from us to BlueZ	<code>RegisterApplication</code>
A signal we emit ourselves	<code>PropertiesChanged</code> from <code>emit_led_status()</code>

main.c — Putting It All Together

main.c

```
int main()
{
    GMainLoop *loop;

    device_manager_init();

    adapter_print_name();

    signal_handler_init();

    gatt_server_init();
    advertisement_init();

    adapter_start_discovery();

    loop = g_main_loop_new(NULL, FALSE);
    g_main_loop_run(loop);
    g_main_loop_unref(loop);

    return 0;
}
```

Every function name here should now be familiar — this chapter is really just confirming the order, which matters more than it might look:

1. `device_manager_init()` clears the device table (Chapter 7) before anything can possibly try to fill it.
2. `adapter_print_name()` is a simple sanity check (Chapter 6) — if this fails, nothing else stands a chance either, since it's the first real D-Bus round trip.
3. `signal_handler_init()` subscribes to BlueZ's signals (Chapter 8) *before* discovery starts, so no early device announcement can be missed.
4. `gatt_server_init()` and `advertisement_init()` (Chapters 10 and 9) set up the Peripheral side. Either order would work here since they don't depend on each other, but the GATT service is registered first so that, by the time we start advertising it a moment later, the service already exists.
5. `adapter_start_discovery()` (Chapter 6) is called last among the setup steps, precisely because the signal handler above is already listening by this point.

11.1 The main loop: why the program doesn't just exit

Every function above either returns almost immediately or, in the case of the asynchronous D-Bus calls in Chapters 9 and 10, schedules a callback for later. None of them block forever. Without something to keep the process alive, `main()` would reach its closing `return 0;` within milliseconds and the whole program would exit before a single Bluetooth signal or callback ever had a chance to fire.

`g_main_loop_run(loop)` is that "something": it's GLib's event loop, and it blocks — deliberately and forever (until something calls `g_main_loop_quit()`, which nothing in this project currently does) — continuously waiting for and dispatching whatever comes in: D-Bus signals, D-Bus method call replies, incoming method calls from BlueZ, and the 3-second LED timer from Chapter 10. Every single callback function you've read about in this book runs *inside* this loop, one at a time, from this one line.

NOTE This single-threaded, event-driven design is why none of this project's callback functions need locks or mutexes around the device table or the LED state — GLib guarantees only one callback runs at a time.

Building, Running, and Testing With Your Phone

12.1 What you need

- A Linux machine with a Bluetooth adapter (a built-in laptop adapter, a USB dongle, or an embedded board like a Raspberry Pi) and **BlueZ** installed (it ships by default on almost every mainstream distribution).
- The GLib development headers, since this project links against GLib/GIO — on Debian/Ubuntu-family systems this is the package usually named `libglib2.0-dev`.
- A phone with a free BLE scanning app installed, such as **nRF Connect for Mobile** (Nordic Semiconductor).

12.2 Building

Compile every `.c` file together, linking against GLib/GIO (`pkg-config` resolves the right flags automatically). From the directory containing all the source files:

build command

```
gcc *.c $(pkg-config --cflags --libs glib-2.0 gio-2.0) -o ble_app
```

12.3 Permissions

Talking to BlueZ over the **system** D-Bus bus (Chapter 3), registering a GATT application, and controlling discovery are all privileged operations on most systems. The simplest way to run this project is with elevated privileges:

run command

```
sudo ./ble_app
```

12.4 Before you run it: make sure the adapter is powered on

one-time setup

```
bluetoothctl power on
```

NOTE If you see `RegisterAdvertisement failed` or `RegisterApplication failed` in the logs, this is overwhelmingly the most common cause — both `advertisement.c` and `gatt_server.c` print this exact hint for that reason.

12.5 What you should see in the terminal

On a healthy run, expect log lines in roughly this order: a successful D-Bus connection, the adapter's name, the GATT application and advertisement both registering successfully, then — as soon as any nearby Bluetooth device is in range — a stream of `Device Updated` blocks from Chapter 8 as your own discovery finds other devices, interleaved with an `LED status: ON / OFF` line every three seconds from Chapter 10's timer.

12.6 Finding and testing it from your phone

1. Open nRF Connect, go to the **Scanner** tab, and look for a device advertising the name `BLE-Framework` (set in `advertisement.c`'s `LocalName` property).
2. Tap it, then tap **Connect**.
3. Expand the service list and find the service whose UUID ends in `...def0`. Inside it you should see two characteristics, ending in `...def1` (read/write) and `...def2` (read/notify).
4. On the `...def1` characteristic, tap the read icon — you should see the bytes for `"hello"` the first time. Then tap write, send a short piece of text, and read again — you should get back exactly what you wrote.
5. On the `...def2` characteristic, tap the "subscribe" / notifications icon (this triggers `StartNotify` from Chapter 10). You should see the value flip between `00` and `01` roughly every three seconds, without you doing anything else — that's the LED timer's `PropertyChanged` signal reaching your phone as a live push update.

TRY IT If notifications never arrive but a manual read of the same characteristic shows the value changing, double check that `notifying` actually became `TRUE` in the logs — some apps require you to explicitly tap a "subscribe" bell icon separately from just viewing the characteristic.

Ideas for Extending the Project

With the whole codebase mapped out, here are a few natural next steps, roughly ordered from smallest to most involved — each one exercises a mechanism this book has already fully explained.

- **Add a third characteristic.** Copy the pattern from `char1` (Chapter 10.5) — new UUID constant, new introspection XML, new vtable, one more entry in `build_managed_objects()` — to expose a second sensor value, or a button-press counter.
- **Wire the LED to a real GPIO.** The exact spot to change is called out in a comment inside `led_toggle_timeout_cb()` (Chapter 10.5) — replace it with a real hardware write on a board like the Luckfox Pico Plus mentioned in the project's own headers.
- **Track multiple notification subscribers properly.** Right now `notifying` is a single global flag (Chapter 10.5) — a more complete implementation would track subscriptions per D-Bus sender.
- **Make `properties_changed_callback()` update the device table,** not just log it (a gap noted at the end of Chapter 8), so RSSI and connection-state changes for known devices stay live in `device_table`.
- **Persist `char0`'s value to disk,** so a written value survives a restart of the program.
- **Add a clean shutdown path.** Nothing currently calls `g_main_loop_quit()` (Chapter 11) — hooking `SIGINT` / `SIGTERM` to unregister the advertisement and GATT application before exiting would make the device disappear from scans immediately instead of lingering until it times out.

Glossary of Terms

Advertising	A BLE Peripheral's periodic, connectionless broadcast announcing its presence and basic info.
BlueZ	The official Linux Bluetooth protocol stack, running as the background daemon bluetoothd.
Central	The device that scans for and initiates a connection to a Peripheral. Usually a phone.
Characteristic	A single readable/writable/notifiable piece of data inside a GATT Service.
D-Bus	An inter-process message bus used on Linux for programs to call each other's methods and broadcast signals.
GATT	Generic Attribute Profile — the specification defining how Services and Characteristics are organized once a BLE connection is established.
GDBus / GLib	The C library this project uses to talk to D-Bus, providing types like GDBusConnection and GVariant.
GVariant	GLib's generic container type able to hold any D-Bus data value, described by a type string.
Notify	A Characteristic flag allowing the Peripheral to push new values to a subscribed Central without being asked.
Object path	A hierarchical, string-based address for an object exposed on D-Bus, e.g. /org/bluez/hci0.
Peripheral	The device that advertises and accepts connections. This project plays this role.
PropertiesChanged	A standard D-Bus signal announcing that one or more of an object's properties changed — the exact mechanism this project uses to implement BLE notifications.
Service	A logical grouping of related Characteristics inside a GATT tree, identified by a UUID.
UUID	A 128-bit identifier used to uniquely name a GATT Service or Characteristic.
Vtable (GDBusInterfaceVTable)	A struct of function pointers GDBus calls into when a remote peer invokes a method or reads/writes a property on an object we registered.